

LECTURE 13: TEXT FEATURES, BAG OF WORDS & TF-IDF

Text Preprocessing & Canonicalization

Machine learning models require numeric matrices; raw strings must be mapped to quantitative feature vectors while preserving semantic meaning.

Lowercasing (`s.str.lower()`) ensures casing does not split identical words (e.g. "Chief" and "chief").

Punctuation removal (`r'^[\w\s]+'`) strips trailing commas, periods, and hyphens so abbreviations map to base tokens.

Stop words ("and", "of", "the", "in", "to") are common grammatical words that appear everywhere; removing them prevents counts from being dominated by non-informative terms.

Domain cleaning removes Roman numerals (`r'\b(i|ii|iii|iv|v)\b'`) or job codes; privacy ethics requires redacting PII in public salary data.

Bag of Words (BoW) & Counts Matrix

Represents each document as an unordered collection of word frequencies from a fixed shared vocabulary V .

Constructs matrix $M \in \mathbb{R}^{N \times |V|}$ where $M_{i,j} = \text{count}(w_j \text{ in } d_i)$. Grammar, word order, and negation are lost ("not bad, great" produces the exact same vector as "not great, bad"). Matrices are >99% zeros; stored as SciPy sparse matrices (`csr_matrix`).

Similarity: Dot Product vs. Cosine Similarity

The raw dot product $\vec{u} \cdot \vec{v} = \sum u_j v_j$ scales with document length; two long unrelated essays can have a higher dot product than two short identical summaries.

Cosine similarity measures the angle between vectors, normalizing for length:

$$\text{CosineSim}(\vec{u}, \vec{v}) = \cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|_2 \|\vec{v}\|_2} = \frac{\sum u_i v_i}{\sqrt{\sum u_i^2} \sqrt{\sum v_i^2}}$$

Ranges from 0 (no shared words, orthogonal) to 1 (identical word proportions). Length-invariant: a short summary and long book on the same topic achieve ≈ 1 .

TF-IDF (Term Frequency – Inverse Document Frequency)

Measures how uniquely characteristic word t is to document d within N documents:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \text{idf}(t) = \frac{\text{count}(t \text{ in } d)}{\text{total words in } d} \times \log\left(\frac{N}{\text{df}(t)}\right)$$

$\text{tf}(t, d)$ = local importance in d ; $\text{idf}(t)$ = global rarity across all N documents ($\text{df}(t)$ = docs containing t).

Why log in IDF: (1) Dampens extreme ratios ($N = 10^6$, $\text{df} = 1 \implies 10^6$ dampened to ≈ 13.8 so IDF does not overpower TF); (2) Captures diminishing returns of rarity (2/50 vs 2/500 docs); (3) Cancels universal words ($\text{df} = N \implies \log(1) = 0 \implies \text{tfidf} = 0$).

TF-IDF is 0 if t is not in d ($\text{tf} = 0$) OR if t appears in every single document ($\text{idf} = 0$).

Presidential Addresses example: common rhetoric ("america", "government") gets TF-IDF ≈ 0 , while distinctive terms stand out (Lincoln: "slaves", FDR: "action").

HOW TO EXPLAIN TO THE PROFESSOR

- **Why Cosine Similarity over Euclidean?** Euclidean distance grows with text length even for identical topics; Cosine similarity isolates vector direction (word distribution) from magnitude.
- **Out-of-vocabulary words:** Ignored during `.transform()` because vocabulary matrix dimensions are fixed during `.fit()`.

LECTURE 14: LINEAR REGRESSION & MODEL EVALUATION

Modeling Goals & Empirical Risk Minimization (ERM)

- Models serve two goals: **prediction** ($\hat{y}_{\text{new}} = H(\vec{x}_{\text{new}})$) and **inference** (interpreting weights w_j).
- Residual $e_i = y_i - \hat{y}_i = y_i - H(x_i)$ is signed observation error. Squared loss $L = (y_i - \hat{y}_i)^2$ penalizes large deviations quadratically.
- Mean Squared Error (MSE / Empirical Risk) & Root Mean Squared Error (RMSE):

$$R_{\text{sq}}(H) = \frac{1}{n} \sum_{i=1}^n (y_i - H(x_i))^2 = \frac{1}{n} \sum_{i=1}^n e_i^2 \quad \Bigg| \quad \text{RMSE} = \sqrt{\text{MSE}}$$

- RMSE is expressed in the exact same physical units as response y (e.g. dollars).

Constant Baseline & Simple Linear Regression (SLR)

- Optimal constant model $H(x) = c$ minimizing squared error is sample mean $c^* = \bar{y}$.
- RMSE of constant model is standard deviation $\sigma_y = \sqrt{\frac{1}{n} \sum (y_i - \bar{y})^2}$. A regression model is only useful if its RMSE $< \sigma_y$.
- Simple Linear Regression $H(x) = w_0 + w_1 x$:

$$w_1^* = r \frac{\sigma_y}{\sigma_x} = \frac{\text{Cov}(x, y)}{\sigma_x^2} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \quad \Bigg| \quad w_0^* = \bar{y} - w_1^* \bar{x}$$

- **3 Key SLR Properties:** (1) Line passes through centroid $(\bar{x}, \bar{y})$; (2) Average residual is zero ($\sum e_i = 0$); (3) Residuals are uncorrelated with input x ($\sum x_i e_i = 0$).

Multiple Linear Regression & Normal Equation

- Model with d features: $H(\vec{x}) = w_0 + w_1 x^{(1)} + \dots + w_d x^{(d)} = \vec{x}^T \vec{w}$.
- Design matrix $X \in \mathbb{R}^{n \times (d+1)}$ has leading column of 1s for intercept w_0 .
- Predictions: $\hat{\vec{y}} = X \vec{w}$; Residuals: $\vec{e} = \vec{y} - X \vec{w}$. Normal Equation gives optimal weights:

$$\vec{w}^* = (X^T X)^{-1} X^T \vec{y}$$

- $X^T X$ is invertible iff X has full column rank $d + 1$ (no collinear features and $n > d$).

Model Assessment: R-Squared (R^2) & Residual Plots

- Coefficient of Determination (R^2) = fraction of variance in y explained by model:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} = 1 - \frac{\text{MSE}(H)}{\text{MSE}(\text{mean model})} \quad \Bigg| \quad \text{RMSE} = \sigma_y \sqrt{1 - R^2}$$

- $R^2 = 1 \implies \text{RMSE} = 0$; $R^2 = 0 \implies$ same as predicting mean $\bar{y}$; $R^2 < 0 \implies$ worse than mean (happens on test data). In SLR with intercept, $R^2 = r^2$.
- **Residual Plots:** Homoscedastic random scatter around 0 = good linear fit; U-shape/curvature = non-linear relationship; Funnel = heteroscedasticity.

HOW TO EXPLAIN TO THE PROFESSOR

- **Adding features effect:** Training RMSE always decreases or stays the same (R^2 increases), but test RMSE can increase due to overfitting.
- **Why can test R^2 be negative?** If test MSE exceeds test variance $\text{Var}(y_{\text{test}})$, then $1 - \frac{\text{MSE}}{\text{Var}} < 0$.

LECTURE 15: FEATURE ENGINEERING & TRANSFORMERS

The Role of Feature Engineering

Linear regression is restricted to flat hyperplanes $\vec{w}^T \vec{x}$ directly on input features; real relationships are non-linear, categorical, and multi-scale. Feature engineering transforms raw domain columns X into engineered numerical features $\phi(X)$, allowing linear models to capture curves and categories while keeping optimization linear in weights $\vec{w}$. Drop uninformative features (IDs, constants).

Categorical Encoding: Ordinal vs. Nominal

Ordinal Features (Ordered): Education (HS < BS < MS < PhD), Ratings (Poor < Fair < Good < Great). Map directly to integers preserving order: `df['rating'].map({'Poor':1, 'Fair':2, 'Good':3, 'Great':4})`.
Nominal Features (Unordered): Smoker (Yes/No), Day (Thur, Fri, Sat, Sun). Arbitrary integers (Thur=0, Fri=1, Sat=2, Sun=3) falsely assume Sun > Thur and equal distances.
One-Hot Encoding (OHE): Creates binary 0/1 indicator column $\mathbb{I}(\text{col} = c)$ for each category.
Geometry of OHE: Model $\hat{\text{tip}} = w_0 + w_1 \text{bill} + w_2 \text{size} + w_3 \mathbb{I}(\text{smoker} = \text{Yes})$:

- Non-Smoker: $\hat{\text{tip}} = w_0 + w_1 \text{bill} + w_2 \text{size}$
- Smoker: $\hat{\text{tip}} = (w_0 + w_3) + w_1 \text{bill} + w_2 \text{size}$
- Creates **two parallel hyperplanes** with identical slopes separated by intercept shift w_3 .

Linearization & Quantitative Scaling

Linearization (Horsepower vs. MPG): Raw scatter shows curved decay; linear fit gives high error. Transforming feature to $\log(\text{hp})$ or $1/\text{hp}$ linearizes relationship:
$$\text{mpg} = w_0 + w_1 \log(\text{hp})$$

Flattens curved residuals to random scatter, lowers RMSE, raises R^2 .
`StandardScaler()` standardizes via $z = \frac{x - \mu}{\sigma}$ (mean 0, variance 1). `MinMaxScaler()` rescales to $[0, 1]$. `Binarizer(threshold=t)` maps to binary $\mathbb{I}(x > t)$.
Scaling Nuance: In standard OLS linear regression, standardizing features *does not change predictions, residuals, or R^2* ; it only rescales weights w_j . (Scaling is essential for Ridge/Lasso, PCA, and KNN).

Scikit-Learn Transformer Architecture & Data Leakage

Transformers (Preprocessors): Implement `.fit()`, `.transform()`, `.fit_transform()`.
Estimators (Models): Implement `.fit()`, `.predict()`, `.score()`.
`.fit(X)` learns parameters (μ, σ) in `StandardScaler`; categories in `OneHotEncoder`;
`.transform(X)` applies transformation using stored parameters.
Golden Rule: Never fit a transformer on test data. Always fit on X_{train} , then transform both X_{train} and X_{test} to prevent data leakage.

HOW TO EXPLAIN TO THE PROFESSOR

- Why drop='first' in OneHotEncoder?** Prevents the *Dummy Variable Trap*. Keeping all K dummy columns alongside an intercept creates linear dependence ($\sum \text{dummies} = 1 = \text{intercept}$), making $X^T X$ singular and non-invertible.
- Transformer vs. Estimator:** Transformers preprocess feature matrices ($X \rightarrow X'$) via `.transform`; Estimators learn predictive mappings ($X \rightarrow y$) via `.predict`.

LECTURE 16: PIPELINES, MULTICOLLINEARITY & GENERALIZATION

Pipelines & ColumnTransformers

- `ColumnTransformer` applies specific transformers to specified column subsets (e.g. scaling numerics, one-hot encoding categoricals) and concatenates results.
- `FunctionTransformer(func)` wraps custom Python functions (e.g. `np.log`) into an sklearn transformer.
- `Pipeline` chains sequential transformers with a final model estimator. Calling `pl.fit(X_tr, y_tr)` fits all steps in order; `pl.predict(X_te)` transforms test data using training parameters and predicts, eliminating data leakage.

Multicollinearity & The Dummy Variable Trap

- Multicollinearity occurs when one feature is a linear combination of other features.
- Height Example (Inches x_1 vs. CM $x_2 = 2.54x_1$):
$$\hat{y} = w_0 + w_1 x_1 + w_2 (2.54 x_1) = w_0 + (w_1 + 2.54 w_2) x_1$$

Any pair where $w_1 + 2.54 w_2 = 3$ gives identical predictions and minimal error ($w_1 = 1000, w_2 = -392.5$). Columns of X are linearly dependent $\implies \det(X^T X) = 0 \implies (X^T X)^{-1}$ does not exist.
- Crucial Distinction:** Multicollinearity **does not hurt prediction accuracy or generalization** on similar data; it completely **destroys interpretability** because coefficients become unstable and arbitrarily large.

Generalization & Bias-Variance Decomposition

- Generalization evaluates model performance on unseen population data.
- Underfitting (High Bias):** Model too simple (e.g. line for cubic data); high train RMSE, high test RMSE.
- Overfitting (High Variance):** Model overly complex (e.g. degree 25 poly); near-zero train RMSE, massive test RMSE (memorizes sample noise).
- The Bias-Variance Decomposition of Generalization Risk:**

$$\mathbb{E}[(y_{\text{new}} - H(x_{\text{new}}))^2] = \text{Model Bias}^2 + \text{Model Variance} + \text{Observation Error } \sigma^2$$

- Model Bias** $(\mathbb{E}[H(x)] - f(x))^2$: Error from simplistic assumptions.
- Model Variance** $\mathbb{E}[(H(x) - \mathbb{E}[H(x)])^2] \propto \frac{1}{n}$: Prediction sensitivity to training sample.
- Observation Error** σ^2 : Irreducible natural random measurement noise.
- Sample size $n \uparrow \implies$ variance $\rightarrow 0$ (bias unchanged). Feature count $d \uparrow \implies$ bias $\downarrow$, variance $\uparrow$.

Train-Test Splits & Validation Diagnostics

- Training error reflects *only bias*; test error reflects *both bias and variance*.
- $\text{RMSE}_{\text{train}} \approx \text{RMSE}_{\text{test}}$ (both low) $\implies$ well-calibrated, generalizes well.
- $\text{RMSE}_{\text{train}} \ll \text{RMSE}_{\text{test}}$ $\implies$ overfitting (high variance); collect more n , regularize, drop features.
- $\text{RMSE}_{\text{train}} \approx \text{RMSE}_{\text{test}}$ (both high) $\implies$ underfitting (high bias); feature engineering, non-linear terms.

HOW TO EXPLAIN TO THE PROFESSOR

- Why degree 25 polynomial fails on test data:** Zero bias on training sample, but massive variance; interpolates sample noise rather than population signal.
- Variance formula intuition:** In linear regression, model variance is proportional to $\frac{d}{n}$ (d = features, n = observations).